

Chapter Two

PHP Files and Directories

File Handling in PHP

- Opening and Closing Files
- Files are opened in PHP using the *fopen* command.
- The command takes two parameters, *the file to be opened*, and *the mode* in which to open the file.

Syntax:

```
$fp = fopen("file name", "mode");
```

E.g.

```
$fp = fopen("myfile.txt", "r");
```

- If *fopen* is unable to open the file, it returns 0. This can be used to exit the function with an appropriate message.

E.g.

```
if ( !($fp = fopen("myfile.txt", "r")) )  
    exit("Unable to open the input file.");
```

PHP file modes

- Essentially, file modes give the operating system a mechanism to determine how to handle access requests from other people or scripts and a method to check that you have access and permission to a particular file.
- You need to make three choices when opening a file:
 - You might want to open a file for reading only, for writing only, or for both reading and writing.
 - If writing to a file, you might want to overwrite any existing contents of a file or append new data to the end of the file. You also might like to terminate your program gracefully instead of overwriting a file if the file already exists.
 - If you are trying to write to a file on a system that differentiates between binary and text files, you might need to specify this fact.
- The `fopen()` function supports combinations of these three options.

PHP file modes

'r'	Open for reading only . Start at beginning of file.
'r+'	Open for reading and writing . Start at beginning of file.
'w'	Open for writing only . Remove all previous content, if file doesn't exist, create it.
'w+'	Opens the file for reading and writing only. Places the file pointer at the beginning of the file. and remove the previous content. If files does not exist create it.
'a'	Open a file for writing , but start at END of current content.
'a+'	Open for reading and writing , but start at END and create file if necessary.

PHP file modes

x	Open the file for writing , beginning from the start of the file. If the file already exists, it will not be opened, fopen() will return false, and PHP will generate a warning.
x+	Open the file for writing and reading , beginning from the start of the file. If the file already exists, it will not be opened, fopen() will return false, and PHP will generate a warning.
b	Used in conjunction with one of the other modes. You might want to use this mode if your file system differentiates between binary and text files . Windows systems differentiate; Unix systems do not. The PHP developers recommend you always use this option for maximum portability. It is the default mode.
t	Used in conjunction with one of the other modes. This mode is an option only in Windows systems. It is not recommended except before you have ported your code to work with the b option.

PHP files cont'd

- *Closing a File*
- The **fclose()** function is used to close an open file:
- e.g.

```
<?php  
$file = fopen("test.txt","r");  
//some code to be executed  
fclose($file);  
?>
```

Writing to a file

- You can store data in two basic ways: in flat files or in a database.
- A flat file can have many formats, but in general, when we refer to a flat file, we mean a simple text file.
- If you are dealing with information of any reasonable volume, you will probably want to use a database instead.
- However, flat files have their uses, and in some situations you need to know how to use them.
- File handling is an important part of any web application. You often need to open and process a file for different tasks.
- The processes of writing to and reading from files is very similar to many programming languages.
- Writing data to a file requires three steps:
 - Open the file. If the file does not already exist, you need to create it.
 - Write the data to the file.
 - Close the file.

Writing to a file

- You can write to a file using `fwrite()` method (file write) or `fputs()` (file put string); `fputs()` is an alias to `fwrite()`.
- You call `fwrite()` in the following way:
 - `fwrite($fp, $outputstring);`
- This function call tells PHP to write the string stored in `$outputstring` to the file pointed to by `$fp`.

Example:

```
<?php
```

```
$myfile = fopen("newfile.txt", "w") or die("Unable to open file!");
```

```
$txt = "John Doe\n";
```

```
fwrite($myfile, $txt);
```

```
fclose($myfile);
```

```
?>
```


Writing to a file

e.g2. <?php

```
$myfile = fopen("newfile.txt", "w") or die("Unable to open file!");  
fwrite($fp, "Hello from PHP!");  
fclose($myfile); ?>
```

Example: Lets have a form to accept user input to a file

```
<form action="file.php" method="post">
```

```
First Name:<input type="text" name="fname"><br>
```

```
Second Name:<input type="text" name="lname"><br>
```

```
Email:<input type="text" name="email"><br>
```

```
    <input type="submit" name="submit"  
value="Submit">
```

```
</form>
```

Cont'd

- lets have again a PHP script to write the user inputs to a file

```
<?php
```

```
$fname=$_POST["fname"];
```

```
$lname=$_POST["lname"];
```

```
$mail=$_POST["email"];
```

```
$fh= fopen("guest.txt", "w+");
```

```
if(!$fh)
```

```
    echo "the info is not saved, the file doesn't exists";
```

```
else {
```

```
    fwrite($fh, $fname."\t".$lname."\t".$mail."\t");
```

```
    echo "<br />";
```

```
echo "Ur data is Saved!!"."<br/>";
```

```
    fclose($fh);
```

```
}
```

```
?>
```

Reading from a File

- Similarly, reading data from a file takes three steps:
 - Open the file. If you cannot open the file (for example, if it doesn't exist), you need to recognize this and exit gracefully.
 - Read data from the file.
 - Close the file.
- When you want to read data from a file, you have many choices about how much of the file to read at a time.
- Again, you open the file by using `fopen()`. In this case, you open the file for reading only, so you use the file mode `r`.
- The `fread()` function reads from an open file.
- The first parameter of `fread()` contains the name of the file to read from and the second parameter specifies the maximum number of bytes to read.

Reading from a File

- The **readfile()** function also reads a file and writes it to the output buffer.
- **fgets()** and **fgetc()** are functions used to read single line at a time and single character at a time respectively.
- **feof()** function is used along with these functions to check if the "end-of-file" (EOF) has been reached. The **feof()** function is also useful for looping through data of unknown length.
- Example:

```
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to
open file!");
echo fread($myfile,filesize("webdictionary.txt"));
fclose($myfile);
?>
```

Reading a file line by line

- ▶ The **fgets()** function is used to read a single line from a file.
- ▶ Note: After a call to this function the file pointer has moved to the next line.
- ▶ Example
- ▶ The example below reads a file line by line, until the end of file is reached:

```
<?php
    $file = fopen("guest.txt", "r") or exit("Unable to
    open file!");
//Output a line of the file until the end is reached
    while(!feof($file))
    {
        echo fgets($file). "<br />";
    }
    fclose($file);
?>
```

Reading a file character by character

- ▶ The **fgetc()** function is used to read a single character from a file.
- ▶ **Note:** After a call to this function the file pointer moves to the next character.
- ▶ **Example**
- ▶ The example below reads a file character by character, until the end of file is reached:

```
<?php
```

```
    $file=fopen("welcome.txt","r") or exit("Unable  
to open file!");
```

```
    while (!feof($file))  
    {
```

```
        echo fgetc($file);
```

```
    }
```

```
fclose($file);
```

```
?>
```

PHP Renaming and Delete File

- The rename() function is used to give a new name for your file.
 - rename (“old name”, “new name”).
- In PHP, we can delete any file using unlink() function.
- The unlink() function accepts one argument only: file name
- e.g.

```
<?php
```

```
$del=unlink('mydirectory/data.txt');
```

```
if($del){
```

```
echo "File deleted successfully";
```

```
}else{
```

```
echo "Sorry!";
```

```
}
```

Creating and Removing Directory

- The `mkdir()` and `rmdir()` functions are used to create a new directory and remove the created directory respectively.
 - `mkdir("directory name")`
 - `Rmdir("directory name")`
- The `is_dir()` function is used to check if the directory exists before creating or removing it.
- Example:

```
<?php
```

```
    if(!is_dir("NewDirectory")){  
        mkdir("NewDirectory");  
        echo "the new directory is created!!";  
    }  
?>
```


Creating and Removing Directory

```
<?php
```

```
    if(is_dir("NewDirectory")){
```

```
        rmdir("NewDirectory");
```

```
        echo "the new directory is removed!!";
```

```
    }
```

```
?>
```

- Like in files the **rename ()** function is used to rename the name of directories.

Problems with Using Flat Files

- When a file grows large, working with it can be very slow.
- Searching for a particular record or group of records in a flat file is difficult.
- Dealing with concurrent access can become problematic. With enough traffic on a site, a large group of users may be waiting for the file to be unlocked before they can place their order. If the wait is too long, people will go elsewhere.
- The file processing deals with a file using sequential processing; that is, you start from the beginning of the file and read through to the end. Random access can be difficult because you end up reading the whole file into memory, making the changes, and writing the whole file out again. With a large data file, having to go through all these steps becomes a significant overhead.
- Beyond the limits offered by file permissions, there is no easy way of enforcing different levels of access to data.

How RDBMSs Solve These Problems

- Relational database management systems address all these issues:
- RDBMSs can provide much faster access to data than flat files.
And MySQL, the database system we use in this book, has some of the fastest benchmarks of any RDBMS.
- RDBMSs can be easily queried to extract sets of data that fit certain criteria.
- RDBMSs have built-in mechanisms for dealing with concurrent access so that you, as a programmer, don't have to worry about it.
- RDBMSs provide random access to your data.
- RDBMSs have built-in privilege systems. MySQL has particular strengths in this area.

JSON AND XML

- **JSON (JavaScript Object Notation)**
 - is a lightweight data-interchange format that is easy for humans to read and write and for machines to parse and generate.
 - Is easier and faster alternative to XML.
 - Is language independent format that uses human readable text to transmit data objects.
 - Consists of objects of name/value pairs.
 - Files have the extension. json
- Syntactically, JSON is similar to the code for creating JavaScript objects.
- Due to to this similarity, standard javascript methods can be used to convert JSON data into JavaScript objects.
- The following code snippet depicts an example of JSON:

Cont...

- The following list depicts the similarities between JSON syntax and code for JavaScript object:
 - JSON uses name/value pairs to store data.
 - Commas are used to separate multiple data values.
 - Objects are enclosed within curly braces.
 - Square brackets are used to store arrays.
- The following code depicts how to create name/value pairs:
 “Name”: “Value”

JSON keys must be enclosed within double quotes.

JSON value

- The following code snippet depicts storing different types of value using JSON name/values pairs:

```
"fName": "Jane"    \\Storing string value
"lName": "Doe"     \\Storing string value
"isAlive": true    \\Storing boolean value
"age": 23          \\Storing integer value
"children": []     \\Storing an array
"spouse": null     \\Storing null
```

JSON Object

- A JSON object can include the following types of values:
 - A numeric value
 - A string
 - A Boolean value
 - An array
 - An object
 - A null value

JSON Object

- JSON objects are enclosed within curly braces.
- Similar to JavaScript objects, JSON objects can be used to store multiple name/value pairs.
- The following code depicts storing data in a JSON object:

```
{  
  "fName": "Jane",  
  "lName": "Doe",  
  "isAlive": true,  
  "age": 23,  
  "children": [],  
  "spouse": null  
}
```


JSON Array

- JSON arrays can be created by using square brackets, as shown in the following code snippet:

```
{
  "fName": "Jane",
  "lName": "Doe",
  "isAlive": true,
  "age": 23,
  "ContactNumber": [
    {"type": "Mobile", "Number": "+9198765" },
    {"type": "Office", "Number": "+9124456" }
  ],
  "children": [],
  "spouse": null
}
```

- Since JSON uses the same syntax as that of JavaScript objects, JSON arrays can be accessed in the same way as in JavaScript.

XML

- The markup language developed to add structural and formatting information to data.
- It was designed to be simple enough to be included in any application that language is Standard Generalized Markup Language.
- It was adopted as standard by International Organization for Standardization(ISO).
- XML is used to describe the structure of a document not the way that is presented.
- XML is the recommendation of World Wide Consortium (W3C).

Example Sample XML program

```
<?xml version="1.0"?>
<college>
  <studdetail>
    <regno>0556/14</regno>
    <name>
      <firstname>Abebe</firstname>
      <lastname>bekele</lastname>
    </name>
    <country name="Ethiopia"/>
    <branch>Software Eng</branch>
  </studdetail>
</college>
```

Valid an Well Formed XML

- XML documents may be either *valid* or *well formed*.
- A well formed document is one which follows all of the rules of XML.
- Tags are matched and do not overlap, empty elements are ended properly, and the document contains an XML declaration.
- A valid XML document has its own DTD.
- XML should also conforms the rules set out in the DTD.
- There are many XML parsers that checks the document and its DTD.

XML Elements

- XML documents are composed of *three things* i.e., elements, control information, and entities.
- Most of the markup in an XML document is element markup.
- Elements are surrounded by tags as much as they are in HTML.
- Each document has a single root element which contains all of the other markup.
- **Nesting tags:** Even the simplest XML document has nested tags.
- Unlike HTML these must be properly nested and closed in the reverse of the order in which they were opened.
- Each XML tag has to have a closing tag, again unlike HTML.

Cont...

- **Empty tags:** Some tags are empty, they don't have content.
 - If the content of the tag is missing, it appears as `<content/>`
- **Attributes:** Sometimes it is important that elements have information associated with them without that information becoming a separate element.
- **Case Sensitive:** XML is case sensitive and you must use lower case for your markup.
- **Control Information:** There are *three types* of control information
 - Comments
 - Processing instructions
 - Document type declaration.

Cont...

- **Comments:** XML comments are exactly same as HTML.
- They take the form as
 - **<!-- comment text -->**
- **Processing Instructions:** Processing Instructions are used to control applications.
- One of the processing instructions is
 - **<?xml version="1.0">**
- **Document Type Declarations:** Each XML document has an associated DTD.
- The DTD is usually present in separate file, so that it can be used by many files.
- The statement that includes DTD in XML file is

Cont...

- **Entities:** are used to create small pieces of data which you want to use repeatedly throughout your schema.
- **Example A Complete XML program**

```
<?xml version="1.0"?>
<!DOCTYPE stud SYSTEM "student.dtd">
<college> //Root element
<studdetail>
<regno>0556/014</regno>
<name>
<firstname>Abebe</firstname> // child element
<lastname>Bekele</lastname>
</name>
<country name="Ethiopia"/>
<branch>IT</branch>
</studdetail>
</college>
```


Document Type Definition

- Document type definition have been successfully used in SGML applications for many year.
- DTD are document centric.
- They are well understood.
- There are plenty of tools that support DTD.

```
<!ELEMENT college(studetail+)>
<!ELEMENT studetail(regno, name+, country, branch)>
<!ELEMENT regno(#PCDATA)>
<!ELEMENT name(firstname, lastname)>
<!ELEMENT firstname(#PCDATA)>
<!ELEMENT lastname(#PCDATA)>
<!ELEMENT country(#PCDATA)>
<!ATTLIST country name CDATA #REQUIRED>
<!ELEMENT branch(#PCDATA)>
```

XML Schema

- W3C developed a technology called XML schema which they accepted as a recommendation.
- XML schema is itself an XML application which means when you use it you only need a single grammar and can use your normal XML editor to create it.
- **Example XML Schema for XML document**

```
<?xml version ="1.0" ?>
<xsd:schema xmlns =" http://.....">
<xsd:element name = "college">
<xsd:complexType>
<xsd:sequence>
<xsd:element name = "studetail">
```

Cont...

```
<xsd:element      name      =      "regno"      type      =  
"xsd:string"/>  
<xsd:element name = "name">  
<xsd:complexType>  
<xsd:sequence>  
<xsd:element      name      =      "firstname"  
type="xsd:string"/>  
<xsd:element      name=      "lastname"      type      =  
"xsd:string"/>  
<xsd:element name = "country">  
<xsd:complexType>  
<xsd:attribute      name      =      "Ethiopia"      type=  
"xsd:string"/>
```

Cont...

```
</xsd:element>  
</xsd:sequence>  
</xsd:complexType>  
</xsd:element>  
</xsd:sequence>  
</xsd:complexType>  
</xsd:element>  
</xsd:schema>
```

Presenting XML

- XML documents are presented using Extensible Stylesheet which expresses stylesheets.
- XSL stylesheet are not the same as HTML cascading stylesheets.
- They create a style for a specific XML element, with XSL a template is created.
- XSL basically transforms one data structure to another i.e., XML to HTML.

Cont...

- **Example: the XSL file for the XML document**
- This line must be included in the XML document which reference stylesheet
- `<?xml:stylesheet type = "text/xsl" href = "student.xsl"?.`
- Here goes the XSL file

```
<xsl:stylesheet smlns:xsl ="uri:xsl".  
<xsl:template match="/">  
<html>  
<body>  
<h1> Student Database </h1.  
<xsl:for-each select = "college">  
<xsl:for-each select = "studetail">
```

Cont...

```
<xsl:for-each select = "name">
<xsl:value-of select = "firstname"/>
<xsl:value-of select = "lastname"/>
</xsl:for-each>
<xsl:value-of select="country/@name" />
<xsl:value-of select = "branch"/>
</xsl:for-each>
</xsl:for-each>
</body>
</xsl:template>
</xsl:stylesheet>
```